

Skill -04

Code:

```
bharath_sai_reddy@LAPTOP-1  ×  +  ▾
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define MAX_TEXT 100
#define MAX_ARGS 50

typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_INPUT,
    TOKEN_OUTPUT,
    TOKEN_APPEND,
    TOKEN_BACKGROUND,
    TOKEN_SEMICOLON,
} TokenType;

typedef struct {
    TokenType type;
    char value[MAX_TEXT];
} Token;

typedef struct {
    char command[MAX_TEXT];
    char *args[MAX_ARGS];
    int arg_count;

    char input_file[MAX_TEXT];
    char output_file[MAX_TEXT];

    int append;
    int background;
} Command;

const char *tokenTypeName(TokenType type)
{
    switch (type) {
        case TOKEN_WORD:
            return "WORD";
        case TOKEN_PIPE:
            return "PIPE";
        case TOKEN_INPUT:
            return "INPUT";
        case TOKEN_OUTPUT:
            return "OUTPUT";
        case TOKEN_APPEND:
            return "APPEND";
        case TOKEN_BACKGROUND:
            return "BACKGROUND";
        case TOKEN_SEMICOLON:
            return "SEMICOLON";
        default:
            return "UNKNOWN";
    }
}

/* Tokenize input */
int tokenize(const char *input, Token tokens[])

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Exec
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Just
```



bharath_sai_reddy@LAPTOP-1



GNU nano 7.2

```
/* Tokenize input */
int tokenize(const char *input, Token tokens[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS - 1) {

        if (isspace((unsigned char)input[i])) {
            i++;
            continue;
        }

        /* Pipe */
        if (input[i] == '|') {
            tokens[count].type = TOKEN_PIPE;
            strcpy(tokens[count].value, "|");
            count++;
            i++;
        }

        /* Input redirection */
        else if (input[i] == '<') {
            tokens[count].type = TOKEN_INPUT;
            strcpy(tokens[count].value, "<");
            count++;
            i++;
        }

        /* Output / append */
        else if (input[i] == '>') {
            if (input[i + 1] == '>') {
                tokens[count].type = TOKEN_APPEND;
                strcpy(tokens[count].value, ">>");
                count++;
                i += 2;
            }
            else {
                tokens[count].type = TOKEN_OUTPUT;
                strcpy(tokens[count].value, ">");
                count++;
                i++;
            }
        }

        /* Background */
        else if (input[i] == '&') {
            tokens[count].type = TOKEN_BACKGROUND;
            strcpy(tokens[count].value, "&");
            count++;
            i++;
        }

        /* Semicolon */
        else if (input[i] == ';') {
            tokens[count].type = TOKEN_SEMICOLON;
            strcpy(tokens[count].value, ";");
            count++;
        }
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^ Replace

^K Cut
^U Paste

GNU nano 7.2

```
    else {
        int j = 0;

        tokens[count].type = TOKEN_WORD;

        while (input[i] != '\0' &&
                !isspace((unsigned char)input[i]) &&
                input[i] != '|' &&
                input[i] != '<' &&
                input[i] != '>' &&
                input[i] != '&' &&
                input[i] != ';') {

            if (j < MAX_TEXT - 1) {
                tokens[count].value[j++] = input[i];
            }

            i++;
        }

        tokens[count].value[j] = '\0';
        count++;
    }

    return count;
}

/* Display tokens */
void printTokens(Token tokens[], int count)
{
    printf("\n===== TOKENS =====\n");

    for (int i = 0; i < count; i++) {
        printf("%2d : %-12s : %s\n",
               i + 1,
               tokenTypeName(tokens[i].type),
               tokens[i].value);
    }
}

/* Parse tokens into commands */
int parseCommands(Token tokens[], int count, Command commands[])
{
    int command_count = 0;
    int arg_count = 0;

    memset(commands, 0, sizeof(Command) * MAX_TOKENS);

    for (int i = 0; i < count; i++) {

        /* Normal word */
        if (tokens[i].type == TOKEN_WORD) {

            if (commands[command_count].command[0] == '\0') {
                strcpy(commands[command_count].command,
                       tokens[i].value);
            }

            if (arg_count < MAX_ARGS - 1) {
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

```
bharath_sai_reddy@LAPTOP-1 × + v
GNU nano 7.2
void freeCommands(Command commands[], int count)
{
    for (int i = 0; i < count; i++) {
        for (int j = 0;
            j < commands[i].arg_count;
            j++) {
            free(commands[i].args[j]);
        }
    }
}

int main()
{
    char input[MAX_TEXT * 3];

    Token tokens[MAX_TOKENS];
    Command commands[MAX_TOKENS];

    printf("=====\n");
    printf("      COMMAND PARSER DEMO      \n");
    printf("=====\n");

    printf("\nEnter a command: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Input error\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Tokenize */
    int token_count = tokenize(input, tokens);

    /* Print tokens */
    printTokens(tokens, token_count);

    /* Parse commands */
    int command_count =
        parseCommands(tokens, token_count, commands);

    if (command_count < 0) {
        return 1;
    }

    /* Print parsed commands */
    printCommands(commands, command_count);

    /* Free memory */
    freeCommands(commands, command_count);

    printf("\n=====\n");
    printf("      Parser completed successfully  \n");
    printf("=====\n");

    return 0;
}
|
^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

Output:

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Skill/Skill-04$ gcc tokens_and_commands.c -o tokens_and_commands
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Skill/Skill-04$ ./tokens_and_commands
=====
COMMAND PARSER DEMO
=====

Enter a command: ls -l /home

===== TOKENS =====
1 : WORD      : ls
2 : WORD      : -l
3 : WORD      : /home

===== PARSED COMMANDS =====

Command 1
Command      : ls
Arguments    : ls -l /home
Background   : NO

=====
Parser completed successfully
=====
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Skill/Skill-04$ |
```